Ground Control Station Support for Autonomous Navigation

Project: NavxDrone — Ground Control Station for the ISRO ASCEND Competition **Submitted to:** ISRO ASCEND **Base Platform:** ArduPilot Mission Planner (customized fork, branch `dev/isro`)

Abstract

This report describes the design and implementation of the autonomous navigation support functions of NavxDrone, the ground control station (GCS) developed for the ISRO ASCEND competition. NavxDrone is a customized fork of the open-source ArduPilot Mission Planner, adapted to provide mission planning, real-time autonomous-mode flight control, geofence configuration, and a purpose-built telemetry dashboard for monitoring the vehicle during autonomous operation. The GCS communicates with the flight controller over the MAVLink protocol and provides the operator interface required to plan, command, and supervise autonomous flight — while the autonomous navigation logic itself (path execution, attitude/position control) executes onboard the ArduPilot flight controller.

1. Introduction

Autonomous navigation in an unmanned aerial system is a joint responsibility of the onboard flight controller and the ground control station that plans and supervises the mission. The flight controller (running ArduPilot firmware) executes low-level navigation — waypoint sequencing, loitering, return-to-launch, and geofence enforcement — based on commands and mission data uploaded from the GCS. NavxDrone is the GCS built for this purpose: it allows an operator to author a waypoint mission, upload it to the vehicle, switch the vehicle into an autonomous flight mode, and monitor vehicle state in real time throughout the autonomous flight.

This report documents the specific subsystems of NavxDrone that support autonomous navigation: mission planning, flight-mode control, geofencing, the MAVLink communication link, and the live telemetry dashboard used to supervise autonomous flight.

2. System Architecture

NavxDrone communicates with the flight controller over a serial MAVLink link. The connection is established and managed centrally in `MainV2.cs`, where a `MAVLinkInterface` instance (`comPort`) is created and bound to a `SerialPort` (baud rate configurable, default 57600). All higher-level GCS views — flight planning, flight data, configuration, and telemetry — read vehicle state from and send commands through this shared MAVLink interface.

```
Operator (NavxDrone GCS)
  ■
  ■■ Flight Planner  → uploads waypoint mission (MAVLink mission items)
  ■■ Flight Data      → commands flight mode (AUTO / GUIDED / LOITER / RTL)
  ■■ Configuration    → sets geofence parameters
  ■■ Telemetry Dashboard → reads live vehicle state (CurrentState)
            ■
            ▼
  MAVLinkInterface (ExtLibs/ArduPilot/Mavlink/MAVLinkInterface.cs)
            ■
            ▼
  Flight Controller (ArduPilot firmware) — executes autonomous navigation
```

The flight controller, not the GCS, executes the autonomous navigation control loop. NavxDrone's role is mission authoring, mode supervision, safety-boundary configuration, and real-time situational awareness.

3. Methodology / Implementation

3.1 Mission Planning (Waypoint Autonomous Navigation)

The flight planning interface (`GCSViews/FlightPlanner.cs`) allows the operator to construct an autonomous mission as an ordered sequence of MAVLink mission commands, including:

- `WAYPOINT` and `SPLINE_WAYPOINT` — autonomous point-to-point and curved-path navigation
- `TAKEOFF` and `LAND` — autonomous takeoff and landing
- `RETURN_TO_LAUNCH` — autonomous return to the launch point
- `LOITER_UNLIM`, `LOITER_TURNS`, `LOITER_TIME` — autonomous station-keeping behaviors
- `DO_DIGICAM_CONTROL` — payload (camera) actions synchronized to mission progress

Mission items are added and parameterized (latitude, longitude, altitude) through the planner's `AddCommand()` workflow, then uploaded to the vehicle over MAVLink for autonomous execution.

3.2 Autonomous Flight Mode Control

Once a mission is uploaded, the operator commands the vehicle into an autonomous flight mode from `GCSViews/FlightData.cs` via `setMode()`. Supported autonomous modes include:

- **AUTO** — executes the uploaded waypoint mission
- **GUIDED** — accepts externally-issued autonomous navigation targets in flight
- **LOITER** — autonomous position-hold
- **RTL** — autonomous return-to-launch

`setMode()` translates the requested mode into a MAVLink `DO_SET_MODE` command, dispatched to the flight controller through `MAVLinkInterface.cs`.

3.3 Geofence Configuration

Safety boundaries for autonomous flight are configured through `GCSViews/ConfigurationView/ConfigAC_Fence.cs`, which exposes the flight controller's fence parameters to the operator:

- `FENCE_ENABLE`, `FENCE_TYPE`, `FENCE_ACTION`
- `FENCE_ALT_MAX`, `FENCE_ALT_MIN`, `FENCE_RADIUS`

The configured fence is also rendered as a map overlay on the Flight Data screen, giving the operator a visual boundary reference during autonomous flight.

3.4 Real-Time Telemetry Supervision

A custom telemetry dashboard was added to the Help page (`GCSViews/Help.cs`, `GCSViews/Help.Designer.cs`) to give the operator continuous situational awareness during autonomous operation. A timer-driven update loop refreshes the display every 500 ms from the vehicle's live MAVLink state (`MainV2.comPort.MAV.cs`), showing:

- Battery remaining (%) and per-cell voltages (cells 1–3)
- Battery temperature
- Altitude, pitch, and roll
- Landed state
- Time in air and time since arm

This dashboard allows the operator to detect anomalies (e.g., battery degradation, unexpected attitude, unintended landing) while the vehicle is flying autonomously, without needing to interpret raw MAVLink data.

4. Results

The implemented GCS provides an end-to-end operator workflow for autonomous flight:

1. Author a waypoint mission in the Flight Planner and upload it over MAVLink.
2. Configure geofence safety boundaries appropriate to the operating area.
3. Arm the vehicle and switch to AUTO mode to begin autonomous mission execution, or use GUIDED/LOITER/RTL as needed during the flight.
4. Monitor the autonomous flight in real time via the telemetry dashboard, with sub-second refresh of critical flight and battery parameters.

This workflow was validated functionally against the existing Mission Planner/ArduPilot MAVLink interface; the custom telemetry dashboard and branding (NavxDrone identity, theme, and iconography via `Utilities/ThemeManager.cs`) were verified to build and display correctly against

live `MAV.comport` data.

5. Conclusion

NavxDrone extends the proven Mission Planner GCS with a focused, ISRO-ASCEND-specific telemetry dashboard while retaining the full mission-planning, mode-control, and geofencing capability needed to plan and supervise autonomous navigation. The GCS's role is to give the operator reliable tools to author autonomous missions, command autonomous flight modes, enforce safety boundaries, and continuously supervise the vehicle's state — complementing the autonomous navigation control executed onboard the ArduPilot flight controller.

6. References

- ArduPilot Mission Planner source repository (this codebase, branch `dev/isro`)
- MAVLink protocol specification (mavlink.io)
- ArduPilot firmware documentation (ardupilot.org) — AUTO, GUIDED, LOITER, RTL flight modes and geofence behavior